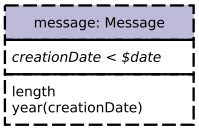
BI / read / 1

BI 1
BI 2
BI 3
BI 4
BI 5
BI 6
BI 7
BI 8
BI 9
BI 10
BI 11
BI 12
BI 13
BI 14
BI 15
BI 16
BI 17
BI 18
BI 19
BI 20

query	BI / read / 1
title	Posting summary
pattern	
desc.	Given a date, find all Messages created before that date. Group them by a 3-level grouping: 1. by year of creation 2. for each year, group into Message types: is Comment or not 3. for each year-type group, split into four groups based on length of their content • `0`: $0 \leq$ length < 40 (short) • `1`: $40 \leq$ length < 80 (one liner) • `2`: $80 \leq$ length < 160 (tweet) • `3`: $160 \leq$ length (long)
params	1 `date` Date

result

1	`year`	32-bit Integer	R	`year(message.creationDate)`
2	`isComment`	Boolean	M	True for Comments, False for Posts
3	`lengthCategory`	32-bit Integer	C	`0` for short, `1` for one-liner, `2` for tweet, `3` for long
4	`messageCount`	32-bit Integer	A	Total number of Messages in that group
5	`averageMessageLength`	32-bit Integer	A	Average length of the Message content in that group
6	`sumMessageLength`	32-bit Integer	A	Sum of all Message content lengths
7	`percentageOfMessages`	32-bit Float	A	Number of Messages in group as a percentage of all messages created before the given date

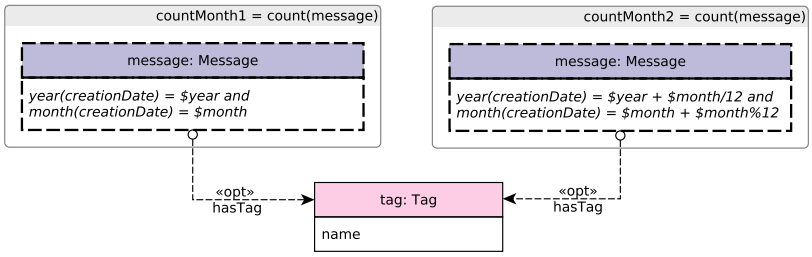
sort

1	`year`	↓	
2	`isComment`	↑	False < True, i.e. Posts come first and Comments second
3	`lengthCategory`	↑	order based on the `lengthCategory` value

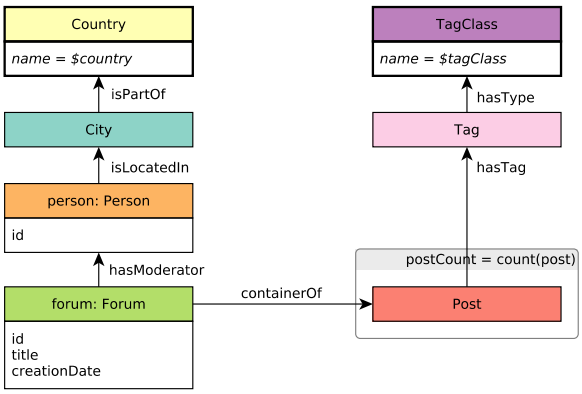
CPs	1.2, 3.2, 4.1, 4.2, 8.5

BI / read / 2

BI 1
BI 2
BI 3
BI 4
BI 5
BI 6
BI 7
BI 8
BI 9
BI 10
BI 11
BI 12
BI 13
BI 14
BI 15
BI 16
BI 17
BI 18
BI 19
BI 20

query	BI / read / 2
title	Tag evolution
pattern	
desc.	Find the Tags that were used in Messages during the given `month` of the given `year` and the Tags that were used during the next month. For the Tags and for both months, compute the count of Messages.
params	1 `year` 32-bit Integer 2 `month` 32-bit Integer
result	1 `tag.name` Long String R 2 `countMonth1` 32-bit Integer A Occurrences of the tag during the given `year` and `month` 3 `countMonth2` 32-bit Integer A Occurrences of the tag during the next month after the given `year` and `month` 4 `diff` 32-bit Integer A Absolute difference of `countMonth1` and `countMonth2`
sort	1 `diff` ↓ 2 `tag.name` ↑
limit	100
CPs	2.4, 3.1, 3.2, 4.1, 4.2, 4.3, 5.3, 6.1, 8.2, 8.5

BI / read / 3

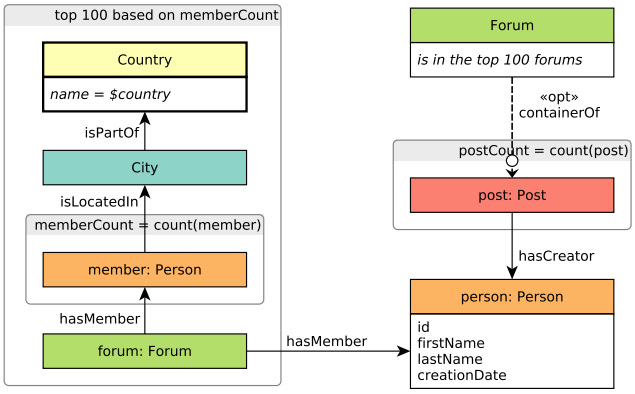
query	BI / read / 3
title	Popular topics in a country
pattern	
desc.	Given a TagClass and a Country, find all the Forums created in the given Country, containing at least one Post with Tags belonging directly to the given TagClass. The location of a Forum is identified by the location of the Forum's moderator.
params	
result	
sort	
limit	20
CPs	1.1, 1.2, 1.3, 2.1, 2.2, 2.4, 3.3, 8.2

1	tagClass	Long String	
2	country	Long String	

1	forum.id	ID	R	
2	forum.title	Long String	R	
3	forum.creationDate	DateTime	R	
4	person.id	ID	R	
5	postCount	32-bit Integer	A	

1	postCount	↓	
2	forum.id	↑	

BI / read / 4

query	BI / read / 4
title	Top posters in a country
pattern	
desc.	Find the most popular Forums for a given Country, where the popularity of a Forum is measured by the number of members that Forum has from the given Country. Calculate the top 100 most popular Forums. In case of a tie, the Forum(s) with the smaller id value(s) should be selected. For each member Person of the 100 most popular Forums, count the number of Posts (postCount) they made in any of those (most popular) Forums. Also include those member Persons who have not posted any Posts (have a postCount of 0).

params

1	country	Long String	

result

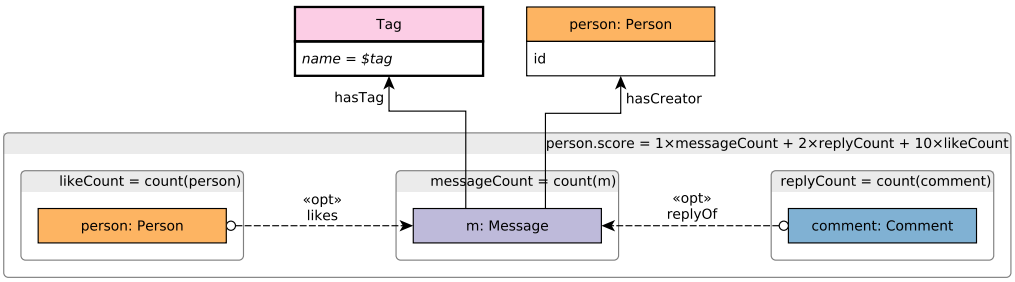
1	person.id	ID	R	
2	person.firstName	String	R	
3	person.lastName	String	R	
4	person.creationDate	DateTime	R	
5	postCount	32-bit Integer	A	

sort

1	postCount	↓	
2	person.id	↑	

limit	100
CPs	1.2, 1.3, 2.1, 2.2, 2.3, 2.4, 3.3, 5.3, 6.1, 8.2, 8.4

BI / read / 5

query	BI / read / 5
title	Most active posters of a given topic
pattern	
desc.	Get each Person (`person`) who has created a Message (`message`) with a given Tag (direct relation, not transitive). Considering only these messages, for each Person node: • Count its `messages` (`messageCount`). • Count likes (`likeCount`) to its `messages`. • Count Comments (`replyCount`) in reply to it `messages`. The `score` is calculated according to the following formula: $1 \times$ `messageCount` $+ 2 \times$ `replyCount` $+ 10 \times$ `likeCount`.

params

1	tag	Long String	

result

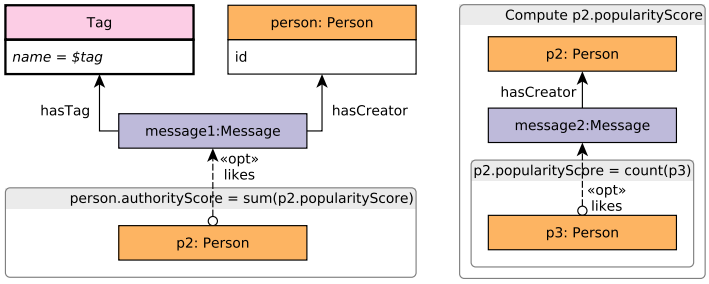
1	person.id	ID	R	
2	replyCount	32-bit Integer	A	
3	likeCount	32-bit Integer	A	
4	messageCount	32-bit Integer	A	
5	score	32-bit Integer	A	

sort

1	score	↓	
2	person.id	↑	

limit	100
CPs	1.2, 2.3, 8.2

BI / read / 6

query	BI / read / 6
title	Most authoritative users on a given topic
pattern	
desc.	Given a Tag (`tag`), find all Persons (`person`) that ever created a Message with the Tag. For each of these Persons (`person`) compute their "authority score" as follows: • The "authority score" is the sum of "popularity scores" of the Persons (`p2`) that liked any of that Person's Messages with the given Tag (same criterion as for `message1`). • A Person's (`p2`) "popularity score" is defined as the total number of likes on all of their Messages (`message2`).

params

1	tag	Long String

result

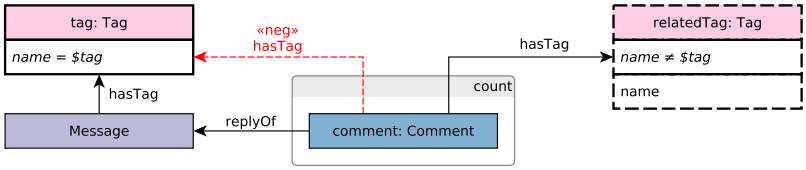
1	person.id	ID	R
2	authorityScore	32-bit Integer	A

sort

1	authorityScore	↓
2	person1.id	↑

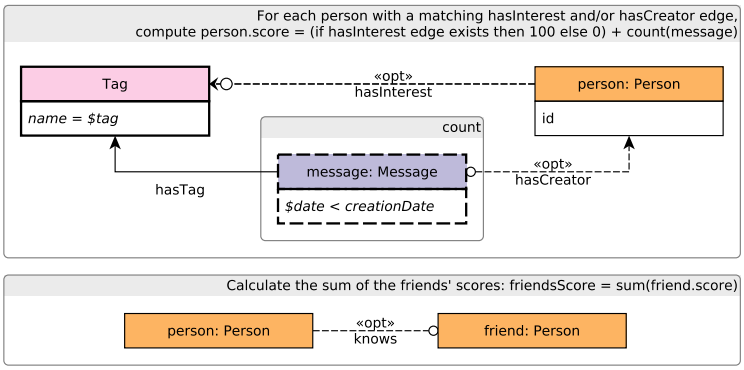
limit	100
CPs	1.2, 2.3, 3.3, 6.1, 8.2

BI / read / 7

query	BI / read / 7
title	Related topics
pattern	
desc.	Find all Messages that have a given Tag. Find the related Tags attached to (direct) reply Comments of these Messages, but only of those reply Comments that do not have the given Tag. Group the Tags by name, and get the count of replies in each group.
params	1 | tag | Long String
result	1 | relatedTag.name | Long String | R 2 | count | 32-bit Integer | A
sort	1 | count | ↓ 2 | relatedTag.name | ↑
limit	100
CPs	1.4, 3.3, 5.2, 8.1

BI / read / 8

BI 1
BI 2
BI 3
BI 4
BI 5
BI 6
BI 7
BI 8
BI 9
BI 10
BI 11
BI 12
BI 13
BI 14
BI 15
BI 16
BI 17
BI 18
BI 19
BI 20

query	BI / read / 8
title	Central person for a tag
pattern	
desc.	Given a Tag, find all Persons that are interested in the Tag and/or have written a Message (Post or Comment) with a `creationDate` after a given `date` and that has a given Tag. For each Person, compute the `score` as the sum of the following two aspects: • 100, if the Person has this Tag as their interest, or 0 otherwise • number of Messages by this Person with the given Tag Also, for each Person, compute the sum of the score of the Person's friends (`friendsScore`).

params

1	tag	Long String
2	date	Date

result

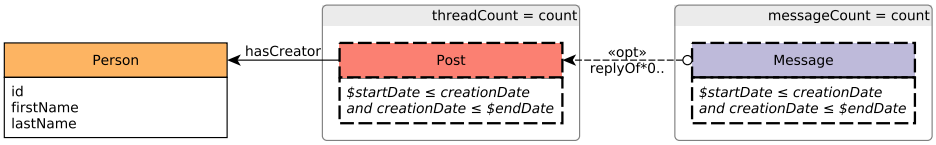
1	`person.id`	ID	R	
2	`score`	32-bit Integer	A	
3	`friendsScore`	32-bit Integer	A	The sum of the score of the `person`'s friends

sort

1	`score + friendsScore`	↓
2	`person.id`	↑

limit	100
CPs	1.2, 2.1, 2.3, 3.2, 5.3, 8.2, 8.4, 8.5
relevance	Similarly to BI Q16, there are two major ways to compute this query: (1) creating an induced subgraph of the interested Persons and their friends and performing the scoring on this graph or (2) performing the scoring without creating an induced subgraph and scoring the friends of a Person on-the-fly. The latter approach is easier to express in systems which do not provide graph views but might result in redundant computations.

BI / read / 9

query	BI / read / 9
title	Top thread initiators
pattern	
desc.	For each Person, count the number of Posts they created in the time interval `[startDate, endDate]` (equivalent to the number of threads they initiated) and the number of Messages in each of their (transitive) reply trees, including the root Post of each tree. When calculating Message counts only consider messages created within the given time interval. Return each Person, number of Posts they created, and the count of all Messages that appeared in the reply trees (including the Post at the root of tree) they created.

params

1	`startDate`	Date	
2	`endDate`	Date	

result

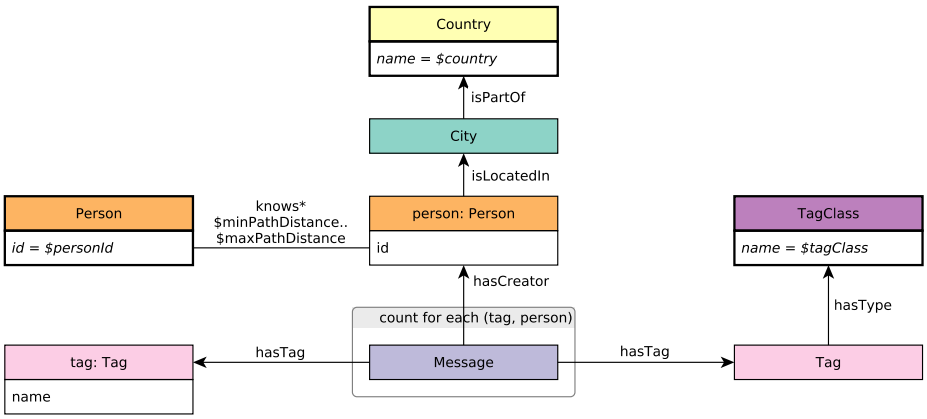
1	`person.id`	ID	R	
2	`person.firstName`	String	R	
3	`person.lastName`	String	R	
4	`threadCount`	32-bit Integer	A	The number of Posts created by that Person (the number of threads initiated)
5	`messageCount`	32-bit Integer	A	The number of Messages created in all the threads this Person initiated

sort

1	`messageCount`	↓	
2	`person.id`	↑	

limit	100
CPs	1.2, 2.2, 2.3, 3.2, 7.2, 7.3, 7.4, 8.1, 8.5

BI / read / 10

query	BI / read / 10
title	Experts in social circle
pattern	
desc.	Given a Person, find all other Persons that live in a given Country and are connected to given Person by a *shortest path* with length in range `[minPathDistance, maxPathDistance]` through the knows relation. For each of these Persons, retrieve all of their Messages that contain at least one Tag belonging to a given TagClass (direct relation not transitive). For each Message, retrieve all of its Tags. Group the results by Persons and Tags, then count the Messages by a certain Person having a certain Tag.
params	

1	`personId`	ID	
2	`country`	String	
3	`tagClass`	Long String	
4	`minPathDistance`	32-bit Integer	
5	`maxPathDistance`	32-bit Integer	

result

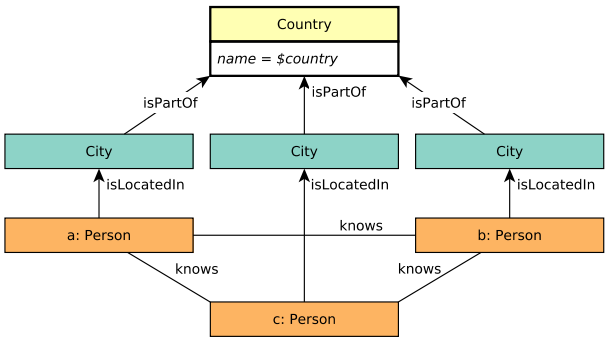
1	`person.id`	ID	R	
2	`tag.name`	Long String	R	
3	`messageCount`	32-bit Integer	A	Number of Messages created by that Person containing that Tag

sort

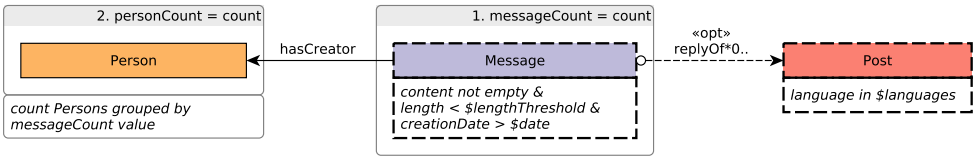
1	`messageCount`	↓	
2	`tag.name`	↑	
3	`person.id`	↑	

limit	100
CPs	1.2, 1.3, 2.3, 2.4, 3.3, 5.3, 7.1, 7.2, 7.3, 8.1, 8.6

BI / read / 11

query	BI / read / 11
title	Friend triangles
pattern	
desc.	For a given `country`, count all the distinct triples of Persons such that: • `a` is friend of `b`, • `b` is friend of `c`, • `c` is friend of `a`. Distinct means that given a triple t_1 in the result set R of all qualified triples, there is no triple t_2 in R such that t_1 and t_2 have the same set of elements.
params	1 | `country` | Long String
result	1 | `count` | 32-bit Integer | A
CPs	1.1, 2.3, 2.5

BI / read / 12

query	BI / read / 12
title	How many persons have a given number of messages
pattern	
desc.	For each Person, count the number of Messages they made (`messageCount`). Only count Messages with the following attributes: • Its `content` is not empty (and consequently, the `imageFile` attribute is empty for Posts). • Its `length` is below the `lengthThreshold` (exclusive, equality is not allowed). • Its `creationDate` is after `date` (exclusive, equality is not allowed). • It is written in any of the given `languages`. – The language of a Post is defined by its `language` attribute. – The language of a Comment is that of the Post that initiates the thread where the Comment replies to. The Post and Comments in the reply tree's path (from the Message to the Post) do not have to satisfy the constraints for `content`, `length` and `creationDate`. For each `messageCount` value, count the number of Persons with exactly `messageCount` Messages (with the required attributes).

params

1	`date`	Date	
2	`lengthThreshold`	32-bit Integer	
3	`languages`	{String}	

result

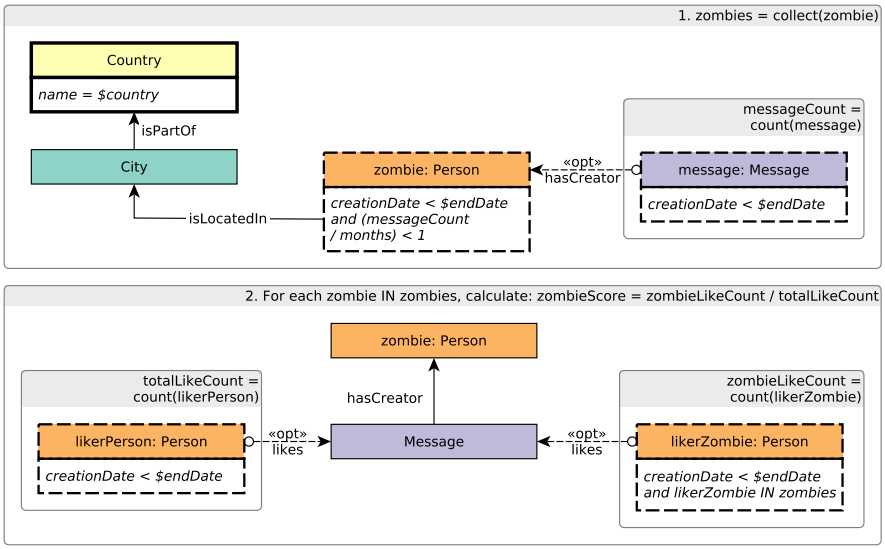
1	`messageCount`	32-bit Integer	A	Number of Messages created
2	`personCount`	32-bit Integer	A	Number of Persons with `messageCount` messages

sort

1	`personCount`	↓	
2	`messageCount`	↓	

CPs	1.1, 1.2, 1.4, 3.2, 4.2, 4.3, 8.1, 8.2, 8.3, 8.4, 8.5

BI / read / 13

query	BI / read / 13
title	Zombies in a country
pattern	
desc.	Find zombies within the given `country`, and return their zombie scores. A `zombie` is a Person created before the given `endDate`, which has created an average of `[0, 1)` Messages per month, during the time range between profile's `creationDate` and the given `endDate`. The number of months spans the time range from the `creationDate` of the profile to the `endDate` with partial months on both end counting as one month (e.g. a `creationDate` of Jan 31 and an `endDate` of Mar 1 result in 3 months). For each `zombie`, calculate the following: • `zombieLikeCount`: the number of likes received from other zombies. • `totalLikeCount`: the total number of likes received. • `zombieScore`: `zombieLikeCount` / `totalLikeCount`. If the value of `totalLikeCount` is 0, the `zombieScore` of the `zombie` should be 0.0. For both `zombieLikeCount` and `totalLikeCount`, only consider likes received from profiles that were created before the given `endDate`.

params

1	country	Long String	
2	endDate	Date	

result

1	zombie.id	ID	R	
2	zombieLikeCount	32-bit Integer	A	
3	totalLikeCount	32-bit Integer	A	
4	zombieScore	64-bit Float	A	zombieLikeCount / totalLikeCount

sort

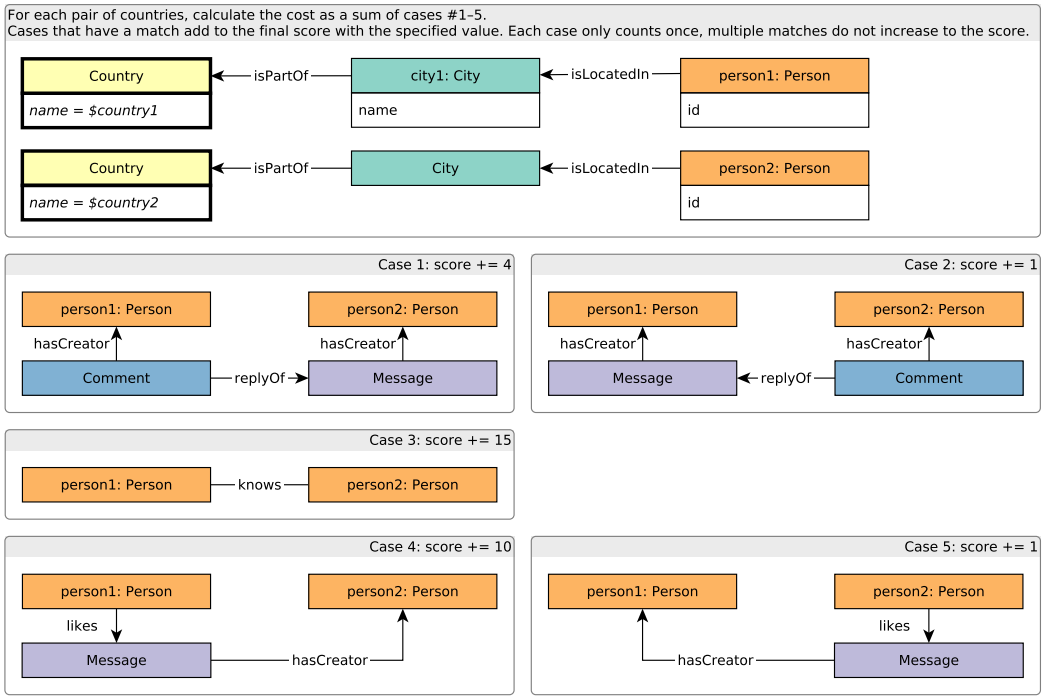
1	zombieScore	↓	
2	zombie.id	↑	

limit	100
CPs	1.2, 2.1, 2.3, 2.4, 3.2, 3.3, 4.2, 5.1, 5.3, 8.2, 8.4, 8.5

BI / read / 14

query	BI / read / 14
title	International dialog

pattern



desc.

Consider all pairs of people `(person1, person2)` such that one is located in a City of Country `country1` and the other is located in a City of Country `country2`. For each City of Country `country1`, return the highest scoring pair. The score of a pair is defined as the sum of the subscores awarded for the following kinds of interaction. The initial value is `score = 0`.

1. `person1` has created a reply Comment to at least one Message by `person2`: `score += 4`
2. `person1` has created at least one Message that `person2` has created a reply Comment to: `score += 1`
3. `person1` and `person2` know each other: `score += 15`
4. `person1` liked at least one Message by `person2`: `score += 10`
5. `person1` has created at least one Message that was liked by `person2`: `score += 1`

Consequently, the maximum score a pair can obtain is: `4 + 1 + 15 + 10 + 1 = 31`.
To break ties, order by (1) `person1.id` ascending and (2) `person2.id` ascending.

params

1	country1	Long String	
2	country2	Long String	

result

1	person1.id	ID	R	
2	person2.id	ID	R	
3	city1.name	Long String	R	
4	score	32-bit Integer	C	

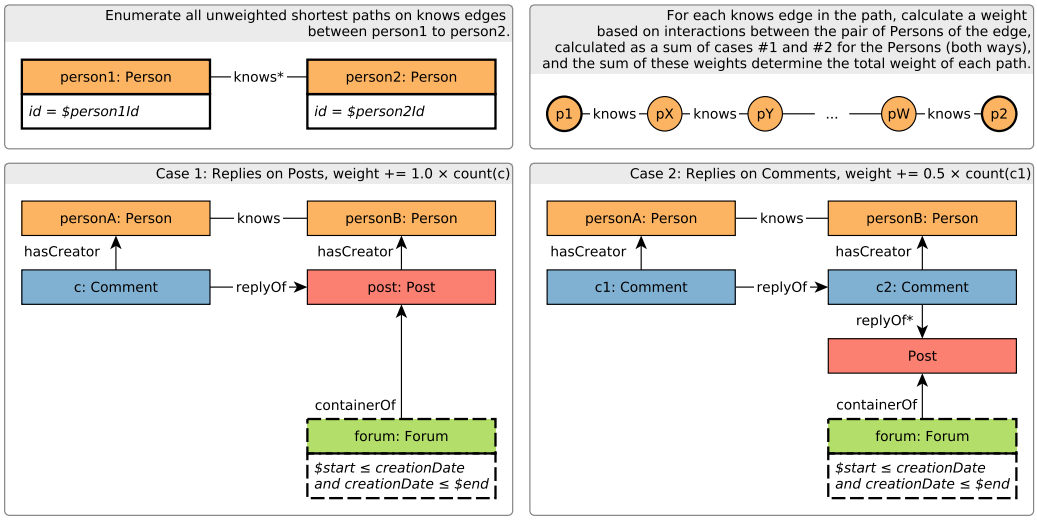
sort

1	score	↓	
2	person1.id	↑	
3	person2.id	↑	

CPs

1.3, 1.4, 2.1, 3.1, 3.3, 5.1, 5.2, 5.3, 8.3, 8.4

BI / read / 15

query	BI / read / 15
title	Trusted connection paths through forums created in a given timeframe
pattern	

desc.	Given two Persons, find all (unweighted) shortest paths between these two Persons, in the subgraph induced by the knows relationship. Then, for each path calculate a weight. The nodes in the path are Persons, and the weight of a path is the sum of weights between every pair of consecutive Person nodes in the path. The weight for a pair of Persons is calculated based on their interactions: • Every direct reply (by one of the Persons) to a Post (by the other Person) contributes 1.0. • Every direct reply (by one of the Persons) to a Comment (by the other Person) contributes 0.5. Only consider Messages that were created in a Forum that was created within the timeframe (interval) `[startDate, endDate]`. Note that for Comments, the containing Forum is that of the Post that the comment (transitively) replies to. Also note that interactions are counted both ways. Return all paths with the Person IDs ordered by their weights descending.

params

1	`person1Id`	ID	
2	`person2Id`	ID	
3	`startDate`	Date	
4	`endDate`	Date	

result

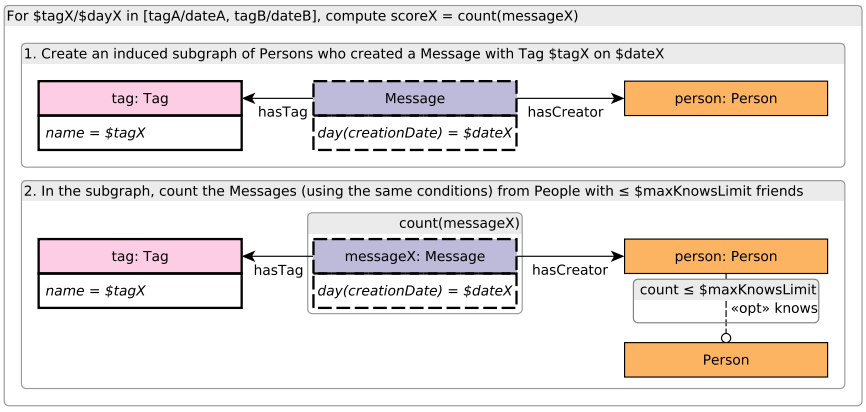
1	`person.id`	[ID]	C	Ordered sequence of the Person IDs in the path
2	`weight`	64-bit Float	C	

sort

1	`weight`	↓	The order of paths with the same weight is unspecified
2	`personIds`	↑	The IDs in the paths are used for lexicographical sorting

CPs	1.2, 2.1, 2.2, 2.4, 3.3, 5.1, 5.3, 7.2, 7.3, 7.5, 7.7, 8.1, 8.2, 8.3, 8.4, 8.5, 8.6

BI / read / 16

query	BI / read / 16
title	Fake news detection
pattern	
desc.	Given two Tag/date pairs (`tagA`/`dateA` and `tagB`/`dateB`), for each pair `tagX`/`dateX`: • Create an induced subgraph between Persons where for each pair of Persons `person1`/`person2`, both have created a Message on the day of `dateX` with Tag `tagX`. • In the induced subgraph, only keep pairs of Persons who have at most `maxKnowsLimit` friends (in the induced subgraph). • For these Persons, count the number of Messages created on `dateX` with Tag `tagX`. Return Persons who had at least one messages for both `tagA`/`dateA` and `tagB`/`dateB` ranked by their total number of messages (descending).

params

1	`tagA`	Long String	
2	`dateA`	Date	
3	`tagB`	Long String	
4	`dateB`	Date	
5	`maxKnowsLimit`	32-bit Integer	

result

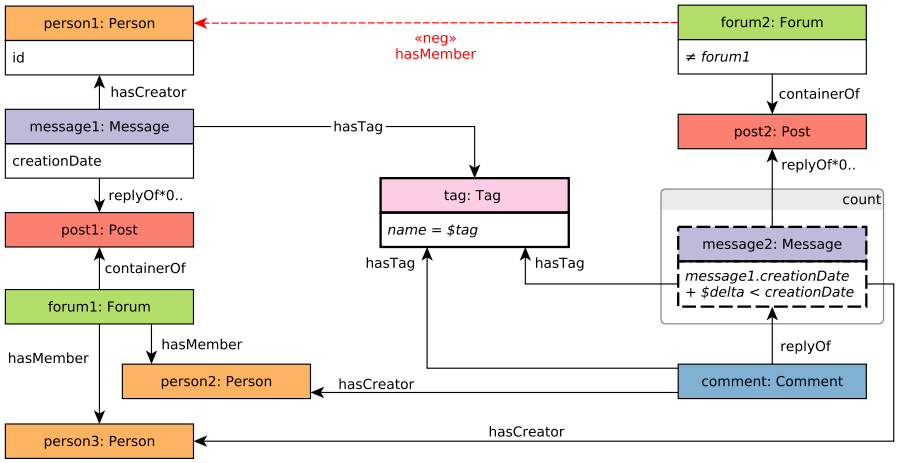
1	`person.id`	ID	R	
2	`messageCountA`	32-bit Integer	A	message count for `tagA`/`dateA`
3	`messageCountB`	32-bit Integer	A	message count for `tagB`/`dateB`

sort

1	`messageCountA` + `messageCountB`	↓	
2	`person.id`	↑	

limit	20
CPs	5.3, 8.4, 8.5
relevance	There are two major ways to compute this query: (1) create the induced subgraph as suggested by the specification (either as a view or in materialized form), or (2) skip creating the induced subgraph and perform on-the-fly check for the number of friends (who also posted at least one Message with the given Tag on the given date). The latter approach is easier to express in systems which do not provide graph views but might result in redundant computations (the query engine will might repeatedly check whether a Person has at least one Message that satifies the conditions).

BI / read / 17

query	BI / read / 17
title	Information propagation analysis
pattern	
desc.	This query aims to identify instances of "information propagation" when a Person (`person1`) submits a Message (`message1`) with a given Tag (`tag`) to a Forum (`forum1`). This is read by other members of `forum1`, Persons `person2` and `person3`. Some time later (specified by the `delta` parameter), these persons have a discussion with the same `tag` in a different Forum (`forum2`) where `person1` is not a member. The discussion consists of a Message (`message2`) by `person2` and a direct reply Comment (`comment`) by `person3`. Return IDs of `person1` with the number of interactions their Messages (might have) caused.
params	

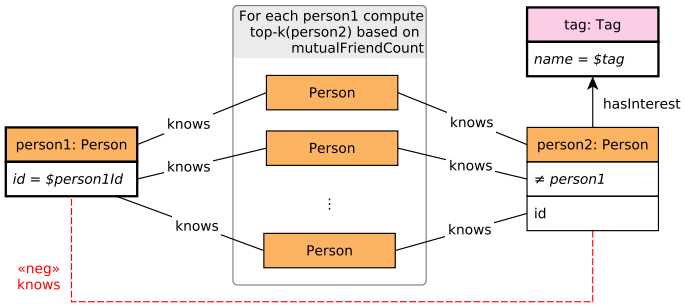
1	tag	Long String	
2	delta	32-bit Integer	Measured in hours

result

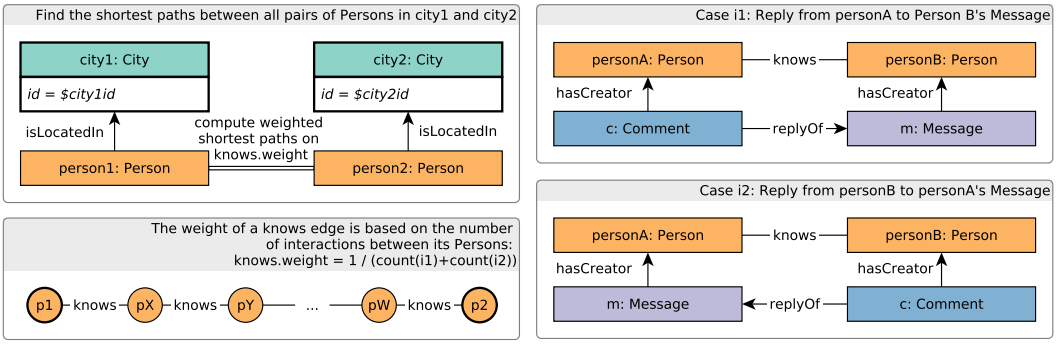
1	person1.id	ID	R	
2	messageCount	32-bit Integer	A	

CPs	2.1, 2.3, 8.1

BI / read / 18

query	BI / read / 18
title	Friend recommendation
pattern	
desc.	For a given Person (`person1`) and a Tag (`tag`), recommend new friends (`person2`) who • do not yet know `person1` • have many mutual friends with `person1` • are interested in `tag`. Rank Persons `person2` based on the number of mutual friends.
params	1 `person1Id` ID 2 `tag` Long String
result	1 `person2.id` ID R 2 `mutualFriendCount` 32-bit Integer A
sort	1 `mutualFriendCount` ↓ 2 `person2.id` ↑
limit	20
CPs	2.5, 8.1

BI / read / 19

query	BI / read / 19
title	Interaction path between cities
pattern	
desc.	Given two Cities `city1`, `city2`, find Persons `person1`, `person2` living in these Cities (respectively) with the shortest *interaction path* between them. If there are multiple pairs of people with shortest paths having the same total weight, return all of them. The shortest path is computed using a weight between two Persons defined as the reciprocal of the number of interactions (direct reply Comments to a Message by the other Person). Therefore, more interactions imply a smaller weight. *Note:* Interactions are counted both ways, i.e. if Alice writes 2 reply Comments to Bob's Messages and Bob writes 3 reply Comments to Alice's Messages, their total number of interactions is 5.

params

1	`city1Id`	ID
2	`city2Id`	ID

result

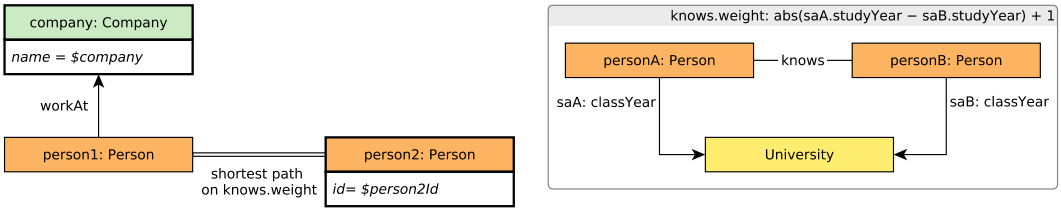
1	`person1.id`	ID	R
2	`person2.id`	ID	R
3	`totalWeight`	32-bit Float	C

sort

1	`totalWeight`	↓
2	`person1.id`	↑
3	`person2.id`	↑

CPs	3.3, 7.6, 7.7, 8.4, 8.6
relevance	Finding shortest paths between pairs of Persons in Cities can be implemented in theory with an *all-pairs shortest paths* algorithm. However, this needs to be executed on the whole Person-knows-Person graph (with edge weights derived from the number of interactions) so it is expected to be prohibitively expensive. A better approach is using multiple *single-source shortest path algorithms* (e.g. from the City with fewer inhibitants).

BI / read / 20

query	BI / read / 20
title	Recruitment
pattern	
desc.	Given a Company `company` and a Person `person2` (who is known to be working at another Company), find a Person (`person1`) working the in the `company` who have the top-20 shortest path to `person2` through people who have studied together. On this path, we only consider edges between Persons who know each other and attended the same university and set the weight of the edge to the absolute difference between the year of enrolment plus 1 (`studyAt.classYear + 1`). If there are multiple Person `person1` nodes with the same shortest path, return all of them.
params	1 `company` Long String 2 `person2Id` ID
result	1 `person1.id` ID R 2 `totalWeight` 32-bit Integer C
sort	1 `person1.id` ↑
CPs	3.3, 7.6, 7.7, 8.4, 8.6
relevance	Implementations can either pre-compute edge weights or compute them on-the-fly. To find a weighted shortest path efficiently, implementations can use e.g. a bidirectional Dijkstra algorithm.